A Cryptanalytic Time-Memory Trade-Off

Martin E. Hellman

October 15, 2026

1 Cryptanalytic Problem with Known Plaintext

We are interested in using a time-space trade-off to break systems containing N possible keys. Here, we assume the attacker can force the system to encrypt a specific, known message, labeled as P_0. The attacker can either try to store all possible values or compute every single one separately, which results in either a massive memory or an massive time delay. Ultimately, we conclude that $O(N^{2/3})$ operations and $O(N^{2/3})$ units of memory provides the most efficient time-space balance.

2 Notations

N: Total number of possible keys in the keyspace.
m: Number of rows in the precomputed matrix.
t: Number of columns (chain length) generated.
M: Memory required.
T: Time required.
K: The unknown key chosen uniformly from the keyspace.
P_0: The chosen plaintext .
C_0: The intercepted ciphertext.
$S_K(P)$: The enciphering operation on plaintext P under key K.
R: A reduction operation mapping the 64-bit ciphertext back down to the 56-bit key space.
$f(K)$: The resulting one-way function defined as $R(S_K(P_0))$.
SP_i: The independently chosen random starting point for the i-th row.
EP_i: The final endpoint of the i-th row.
Y_j: The generated chain elements during the active search phase.

3 Simplifying the Target Function

Let P_0 represent a specific plaintext that the attacker has chosen to feed into the system. The system now encrypts this plaintext using a unknown key K, to produce ciphertext C_0. We denote this by

$$C_0 = S_K(P_0)$$

To build the time-space trade-off the attacker must be able to repeately chain this process. This means feeding the output of one encryption to the key for next encryption. However, the output and the required key input can be different. Hence, we introduce a simple reduction operation R. This function maps the output to the exact size we need for a valid key. Hence, this is denoted by

$$f(K) = R(S_K(P_0))$$

Calculate f(K) in forward direction is fast and simple but working backward to find the input K is not possible. Therefore, f is a one way function. During the search phase of attack, the cryptanalyst intercepts genuine ciphertext C_0 produced by victim. To begin te search, attacker chooses reduction operation R converting valid key to a search target.

$$Y_1 = R(C_0) = f(K)$$

Ultimately, it boils down to invertining the one way function and to discover the key that satisfies $f(K) = Y_1$.

4 Matrix formation

First let us look at 1 row or chain. We select a starting point SP_i. The starting key is random variable chosen from 1 to N. Once they starting key is chosen, the attacker repeaedly applies the one way function to create long sequence of values. Therefore, it can be written as

$$X_{i,j} = f(X_{i,j-1})$$

This continues for the complete chain length of t steps.

They last element produced is designated as endpoint, EP_i which can be expressed as $f^t(SP_i)$. This is where the time space tradeoff takes place. Instead of storing entire sequence of t keys in memory, the attacker only keeps the first and last value. Hence, the entire row can be represented as (SP_i, EP_i).

To build full matrix, the attacker generates multiple rows (m rows) using same process. Once all the rows are generated, the resulting (SP_i, EP_i) pairs are sorted numerically by their endpoints (EP_i). This sorting is crucial because it allows the attacker to perform a rapid binary search to instantly check if a value exists in the table.

During search phase, attacker is trying to find if the key exists in the chain. They start by intercepting ciphertext and applying reduction operation to get their initial search target.

$$Y_1 = R(C_0)$$

The attacker first checks if Y_1 exactly matches any endpoint in the table. If it does not, then the target key is not located in the second last column for the matrix. To check earlier columns, the attacker keeps applying f to the obtained result.

$$Y_{j+1} = f(Y_j)$$

If the attacker finds the match where their generated value is equal to one endpoint, they can find out the key's exact location. Since, they had to walk lets say j steps forward to reach the end, the actual key is located j steps behind the end. To retrieve it the attacker looks up to starting point SP_i for that specific chain and recomputes by application function $t - j$ times. Hence, we get $X_{i,t-j}$.

5 Matrix Overlap and the Probability of Success

When generating multiple rows, the matrix may contain some overlaps. Because the composite encryption sequence $f(K)$ is modeled as a random function rather than a strict one-to-one permutation, different keys can occasionally produce the same output. If two independent chains merge at any point, they will remain merged for the rest of their length, reducing the total number of unique keys covered by the matrix.

Despite this overlap, we can mathematically guarantee a lower bound for the overall probability of success, $P(S)$.

Theorem: If $f(*)$ is modeled as a random function mapping the keyspace $\{1, 2, \ldots, N\}$ into itself, and the target key K is chosen uniformly from this set, the probability of finding the key is bounded by:

$$P(S) \geqslant \frac{1}{N} \sum_{i=1}^{m} \sum_{j=0}^{t-1} \left[\frac{N - it}{N} \right]^{j+1}$$

Proof Derivation:
Let A denote the specific subset of unique keys covered by the first t columns of the matrix (excluding the endpoints). The probability of success is the expected size of this subset divided by the total keyspace N:

$$P(S) = \frac{E|A|}{N}$$

We can calculate the expected size of A by iterating through every single generated element $X_{i,j}$ and checking if it is a newly discovered key. By using an indicator function, the equation becomes a summation

of probabilities:

$$P(S) = \frac{1}{N} \sum_{i=1}^{m} \sum_{j=0}^{t-1} Pr(X_{i,j} \text{ is new})$$

An element is considered new only if it has not appeared in any previously generated row, or earlier within its own current ro. The probability that a specific element $X_{i,j}$ is new is bounded from below by the joint probability that *all* preceding elements in its specific row (from $X_{i,0}$ up to $X_{i,j}$) are also entirely new:

$$Pr(X_{i,j} \text{ is new}) \geqslant Pr(X_{i,0}, X_{i,1}, \ldots, X_{i,j} \text{ are all new})$$

Applying the chain rule of probability, this joint sequence expands into a product of conditional probabilities

$$Pr(X_{i,0} \text{ is new}) \times Pr(X_{i,1} \text{ is new} | X_{i,0} \text{ is new}) \times \cdots \times Pr(X_{i,j} \text{ is new} | X_{i,0}, \ldots, X_{i,j-1} \text{ are new})$$

.

Let $A_{i,0}$ represent the total set of elements already covered by all the previous rows before starting row i. We can rewrite the conditional probability expansion exactly as a sequence of shrinking fractions

$$\frac{N - |A_{i,0}|}{N} \times \frac{N - |A_{i,0}| - 1}{N} \times \cdots \times \frac{N - |A_{i,0}| - j}{N}$$

Because each row contains at most t different elements, the maximum number of previously covered elements up to row i is it. This restriction means every single fraction in the sequence above must evaluate to a value strictly larger than $(N - it)/N$.

Since there are exactly $j + 1$ factors in this expanded sequence, multiplying their lower bounds together yields.

$$Pr(X_{i,j} \text{ is new}) > \left[\frac{N - it}{N}\right]^{j+1}$$

Finally, substituting this isolated probability bound back into the overall double summation produces the proven lower bound for the entire matrix.

$$P(S) > \frac{1}{N} \sum_{i=1}^{m} \sum_{j=0}^{t-1} \left[\frac{N - it}{N}\right]^{j+1}$$

6 Deriving the Optimal Parameters ($N^{1/3}$)

To understand exactly how the specific values of $m = N^{1/3}$ and $t = N^{1/3}$ are chosen, we must minimize the time and memory requirements of the system. The algorithm requires configuring two core variables for a single table which is number of rows m and the chain length t. To operate efficiently and minimize the matrix overlap discussed previously, the system is designed around a fundamental threshold constraint.

$$mt^2 = N$$

At this specific threshold, the probability of success for a *single* table is approximately mt/N. By substituting our threshold constraint ($N = mt^2$) into this probability equation, we can simplify it.

$$P(S) \approx \frac{mt}{mt^2} = \frac{1}{t}$$

Because a single table only has a $1/t$ chance of containing the target key, the cryptanalyst must generate approximately t separate tables (each using a different reduction mapping R) to push the cumulative success probability toward a guaranteed success.

With t independent tables in play, we can calculate the total structural costs of the system:

- **Total Time (T):** The number of tables (t) multiplied by the maximum search operations required per table (t), which yields $T = t^2$.

- **Total Memory (M):** The number of tables (t) multiplied by the number of stored starting points per table (m), which yields $M = mt$.

To find the most cost-effective operating point, the cryptanalyst minimizes the maximum of either cost by setting the total memory exactly equal to the total search time ($M = T$). Equating the two formulas yields:

$$mt = t^2$$

$$m = t$$

Now, we substitute the relation $m = t$ back into our original threshold constraint ($mt^2 = N$):

$$t(t^2) = N$$

$$t^3 = N$$

Solving for t yields the optimal chain length:

$$t = N^{1/3}$$

Because we previously established that $m = t$, it naturally follows that the optimal number of rows is identical:

$$m = N^{1/3}$$

Finally, substituting these solved parameters back into our total cost equations ($T = t^2$ and $M = mt$) rigorously proves that both the total computational time and total memory are bounded at exactly $N^{2/3}$

7 Bounding the False Alarm Overhead

When a search sequence matches an endpoint ($Y_j = EP_i$), the cryptanalyst must walk the corresponding matrix row from the beginning to verify the key. Because the mapping $f(K)$ acts as a random function, accidental collisions can occur where an incorrect key maps to the target sequence, triggering a false alarm. We must prove that checking these false alarms does not mathematically overwhelm the $O(N^{2/3})$ search time.

Theorem: The expected number of false alarms encountered per table searched, denoted as $E(F)$, is strictly bounded by:

$$E(F) \leqslant \frac{mt(t+1)}{2N}$$

Proof Derivation:
Let $F_{i,j}$ denote the specific event that a false alarm occurs because the j-th generated search element matches the i-th stored endpoint ($Y_j = EP_i$). The total expected number of false alarms is the sum of the probabilities of this event occurring at every possible coordinate in the matrix:

$$E(F) \leqslant \sum_{i=1}^{m} \sum_{j=1}^{t} Pr(F_{i,j})$$

For any specific point (i, j), the false alarm event $F_{i,j}$ can happen in exactly j different ways. These represent the different steps where the incorrect key's sequence could have accidentally merged into the i-th row of the matrix: it could merge immediately (where $f(K)$ equals $f^{t-j+1}(SP_i)$), or merge after one iteration (where $f^2(K)$ equals $f^{t-j+2}(SP_i)$), and so on up to j ways
.
Because the variables preceding the merge are treated as independent random variables uniformly distributed over the entire keyspace $\{1, 2, \ldots, N\}$, the probability of any single, specific merging event happening by pure chance is at most $1/N$.

Since there are j independent ways for the merge to occur, the total combined probability $Pr(F_{i,j})$ is bounded by j/N. Substituting this into our double summation yields:

$$E(F) \leqslant \sum_{i=1}^{m} \sum_{j=1}^{t} \frac{j}{N}$$

The inner summation over the column index j is a standard arithmetic progression that evaluates algebraically to $t(t+1)/2$. Factoring the $1/N$ constant out, we substitute this back into the equation:

$$E(F) \leqslant \sum_{i=1}^{m} \frac{t(t+1)}{2N}$$

Because the remaining expression is entirely independent of the outer row index i, summing this constant value m times simply multiplies it by m, completing the proof:

$$E(F) \leqslant \frac{mt(t+1)}{2N}$$